

Sparse Representation over Learned Dictionary for Document Analysis

DO Thanh Ha

Informatics Departement
Mathematics - Mechanics - Informatics
VNU University of Science

Contents

- ❶ Context and applications of sparse representation
- ❷ Sparse representation and learning dictionary
- ❸ Contributions for document analysis
 - Denoising graphical documents
 - Text/Graphic separation
 - Symbol recognition
 - Spotting symbols
- ❹ Conclusions and futur researches

Context

Finding the relevant information on
the huge amount of data?

Idea: basing on the analyses
the structural information of
document

Methods to consider:

- The hierarchy of physical components (*such as pages, columns, paragraphs, text lines, words, tables, figures*): top-down approaches, bottom-up approaches, and hybrid approaches
- The hierarchy of logical components (*such as titles, authors, affiliations, abstracts, sections*): model-based system, rule-based system, or frame-based system

⇒ We use sparse representation over learned dictionary

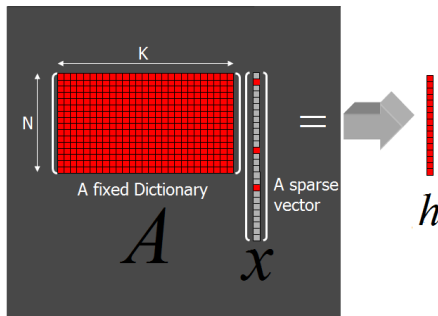
Applications of Sparse Representation

- **Image denoising:** The energy of noise ϵ is chosen according to the (supposed known) standard deviation σ of the noise
- **Image separation:** done by using *Morphological Component Analysis* (MCA) algorithm with two different dictionaries
- **Classification:** done by using sparsity of images, or of local features of images

However

- In denoising: with the specific applications, the noise variance is unknown. How we can overcome this problem?
- In Image separation: what is the best dictionary for each content type? What is the suitable size of the patch?
- In classification: what about the characteristic of the invariance of sparse representations?

Generating Signals in *Sparseland*



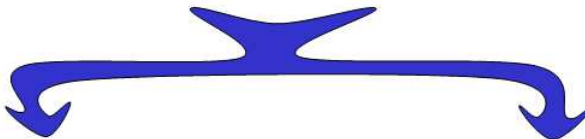
- Every column in A (dictionary) is a prototype signal (atom)
- The vector x is generated randomly with few non-zeros entries in random locations and random values.
- Signal h is built as a linear combination of atoms from the dictionary A

Finding Sparse Representation of Signal

$$\min_x \|x\|_0 \text{ subject to } Ax = h$$

This is a combinatorial problem,
proven to be NP-Hard!

approximation



Greedy Methods

Build the solution one
non-zero element at a time

Relaxation Methods

Smooth the l_0 and use continuous
optimization techniques

Greedy Methods - Matching Pursuits

$$\min_x \|x\|_0 \text{ subject to } Ax = h$$

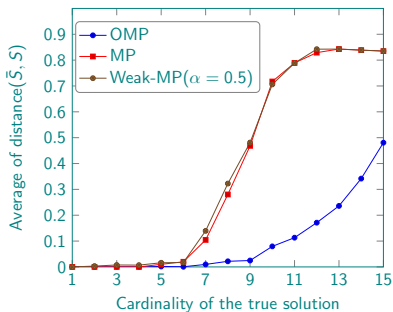
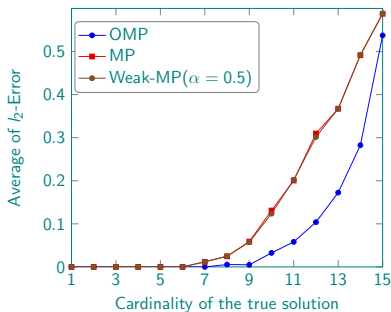
Idea: the column of the dictionary is selected iteratively one by one.

- MP: the selected column is the column a_{i_m} that maximize $|\langle r^m, a_{i_m} \rangle|$ with residue $r^0 = h$
- Weak-MP: the selected column is the column a_{i_m} that satisfies following equation where $\alpha \in (0, 1]$

$$|\langle r^m, a_{i_m} \rangle| \geq \alpha \sup_{j=1, \dots, M} |\langle r^m, a_j \rangle|$$

- Orthogonal-MP: the selected column is the column that is orthogonal to the previously selected columns

Numerical Demonstration



Comment:

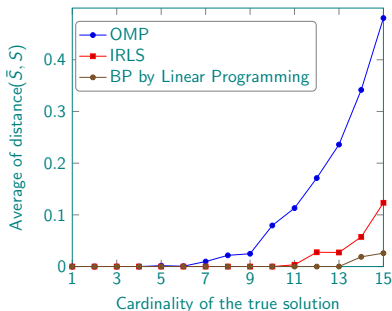
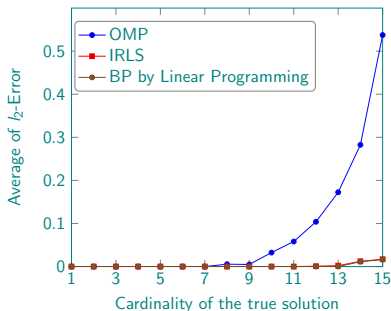
- Overall, all algorithms perform well for low-cardinalities, **the best performing method is the OMP**
- The difference between the MP and Weak MP is quite slight

Relaxation Methods - Basic Pursuits

$$\min_x \|x\|_1 \text{ subject to } Ax = h$$

⇒ solved by IRLS (*Iterative Reweighed Least Squares*) and BP solver linear-programming by Matlab

Idea: rewrite the convex optimization as a linear programming



Numerical Demonstration

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	Average
OMP	0.25	2.89	0.14	0.09	0.11	3.16	0.72	0.21	0.21	0.21	0.66	3.24	0.61	5.74	1.42	2.96
IRLS	13.92	12.78	22.17	13.63	13.32	14.38	14.06	13.64	13.99	13.74	17.19	15.29	15.31	18.44	18.90	15.39
BP by LP	36.59	5.37	4.21	4.52	4.64	3.79	4.40	4.33	4.34	4.49	4.86	4.22	4.42	4.51	4.24	6.60

Table: Time $\times 10^{-2}$ (seconds) performance of difference methods corresponding to the cardinality of the true solution

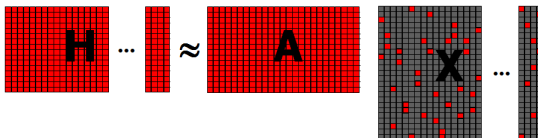
However, the IRLS and BP solver linear-programming by Matlab methods take much more time compared to the OMP

Learning Dictionary - Motivation

- The effect in finding sparse representations of signals depends directly on a dictionary A
- Pre-constructed dictionaries: ridgelets, curvelets, contourlets...
 - Their advantage is the availability of fast transform algorithms
 - However, they are limited to the images that they are designed to handle
- Learning dictionary builds from the underlying empirical data, thus it is able to adapt better to any family of images

Learning Dictionary - Objective

- In a general learning methodology, a family S signals $\{h_j\}_{j=1}^S$ is considered as the training database
- Our goal:** build a dictionary A such as



$\min_{A, x_j}$

$$\sum_{j=1}^S \|x_j\|_1$$

Each signal has
 an optimal sparse
 representation in A

s.t

$$\|h_j - Ax_j\|_2 \leq \epsilon$$

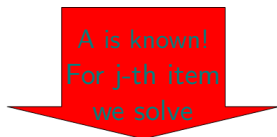
Condition of
 reconstruction error

The Learning Methodology for Construction Dictionary

Such dictionary is obtained by the learning process that iteratively adjusts A via *sparse coding stage* and *update dictionary stage*

Sparse Coding Stage

$$\min_{A, x_j} \sum_{j=1}^S \|x_j\|_1 \text{ s.t. } \|h_j - Ax_j\|_2 \leq \epsilon, \forall j$$



$$\min_{x_j} \|x_j\|_1 \text{ s.t. } \|h_j - Ax_j\|_2 \leq \epsilon$$

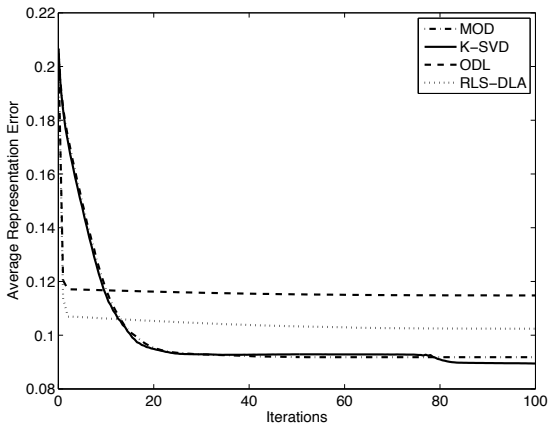
Solved by MP or BP

Update rule in learning algorithms

Update Dictionary Stage: Applying an updating rule to optimize sparse representations of examples

- K-SVD algorithm: Column-by-column of A is updated such as minimizing the residually error based on calculating the SVD (*Singular Value Decomposition*) over the relevant examples
- The MOD (*Method of Optimal Directions*) algorithm: finds an adjustment matrix such as minimization the mean of the set of norm residuals
- The OLD (*Online Learning Dictionary*) algorithm: processes each training signal in each iteration, and uses the warm restart algorithm to update the dictionary
- The RLS-DLA (*Recursive Least Squares*) algorithm: improves the OLD algorithm since updating the dictionary only requires an addition of matrices and a few matrix-by-vector multiplications

Numerical Demonstration



Comment

- Two algorithms K-SVD and MOD behave similarly and comparably, with a small advantage to the K-SVD.
- These two algorithms perform better than ODL and RLS-DLA also

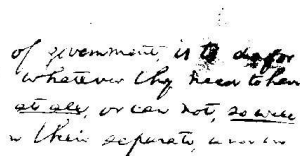
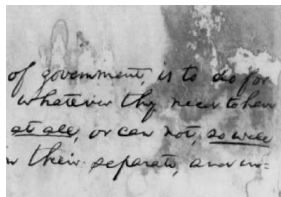
Following numerical experiments, we set the choice:

- OMP algorithm for finding the sparse representation
- K-SVD algorithm for building the dictionary

We apply these choices for all our works in the thesis

Document Noise Removal....

Our first work focuses on denoising document.....



- **Important:** Practical application, such as increasing the performance of pattern recognition techniques applied on images
- **Some methods:** Adaptive filters, Multi Resolution Analysis (MRA) methods with sparse transforms, like curvelets, wavelets,..., Sparse Representations over learned dictionary

Denoising Images by Sparsity: Problem Statement

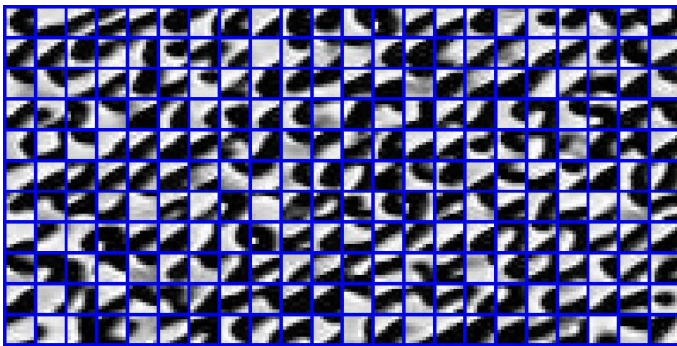
- Let $h_0 = Ax$ be an original image, the noise image $h = h_0 + e$, in which e is the added noise with finite energy $\|e\|_2 \leq \epsilon$
- Let $\bar{x}$ be the solution of following equation:

$$\min_x \|x\|_1 \text{ subject to } \|Ax - h\|_2 \leq \epsilon$$

then the un-known denoised image $\bar{h}$ can be found by $\bar{h} = A\bar{x}$

Denoising Images: Dictionary A

- A is the dictionary that is learned by K-SVD algorithm with training database is the noised patches of noised image



Denoising Images: Finite Energy of Noise

Previous works: ϵ is chosen according to the standard deviation σ for additive white gaussian or the spread relation for noise spread

However, neither of these criteria can be applied to document images where the model of noise is unknown

Thus, we propose a new approach as following:

- Using a dataset including t noisy images I_i^n and corresponding t cleaned images I_i^c ,
- For each couple (I_i^n, I_i^c) , calculating the peak of their normalized cross-correlation matrix, called r_i ,
- Calculating $\bar{r}$ being the mean of all r_i ,
- The value of ϵ is calculated by $\epsilon = c \times \bar{r} \times w$, with c is the constant and w is the size of the patch.

Denoising Images: Algorithm

- ① *Create a training database* using a sliding window of size $w \times w$ to scan the corrupted image $y \in R^{M \times N}$ into $(M - w + 1)(N - w + 1)$ patches $\{h_j\}_{j=1}^S$, $h_j \in R^{w \times w}$
- ② *Create a learned dictionary* using the K-SVD algorithm to create the learned dictionary A from $\{h_j\}_{j=1}^S$.
- ③ Combine the learned dictionary A with the sparse representation model in the purpose of denoising image, following:
 - a. Find the optimal solution $\bar{x}_j$ for each patch h_j
$$\bar{x}_j = \arg \min \|x_j\|_1 \text{ subject to } \|Ax_j - h_j\|_2 \leq \epsilon,$$
 - b. Compute the denoised version of each patch h_j by $\bar{h}_j = A\bar{x}_j$,
 - c. Merge the denoised patches $\bar{h}_j$ to get the denoised image $\bar{y}$.
- ④ Binarise $\bar{y}$ to get the final result $\tilde{y}$.

Experimental Results

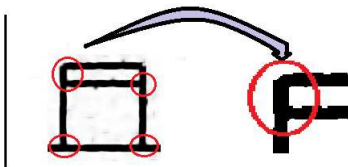
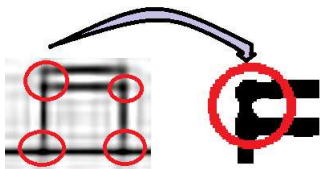
- We evaluate our algorithm on 3 different datasets
 - GREC 2005 dataset: including different symbols divided into 6 levels of noise, and each level contains 50 noised symbols
 - Real scanned documents: 12 scanned datasets with different resolutions are created from 15 original images
 - DIBCO 2009 dataset: containing images that range from grayscale to color and from synthetic to real
- We examine the effectiveness of the proposed method by comparing it with three existing methods: median filtering, morphological filter and curvelet transform
- We perform the statistic test to verify if the difference between the results are significant

Experimental Results with GREC2005 Dataset

	Median	OC	Curvelet	Proposed method
Level 1	0.926 (-)	0.937 (-)	0.963 (=)	0.963
Level 2	0.630 (-)	0.177 (-)	0.565 (-)	0.655
Level 3	0.428 (-)	0.787 (-)	0.389 (-)	0.826
Level 4	0.088 (-)	0.001 (-)	0.459 (+)	0.135
Level 5	0.261 (-)	0.268 (-)	0.263 (-)	0.284
Level 6	0.001 (-)	0.000 (-)	0.059 (=)	0.060

	Median	OC	Curvelet	Proposed method
Level 1	2.896 (-)	2.454 (-)	1.403 (=)	1.423
Level 2	21.015 (-)	30.865 (-)	28.421 (-)	19.687
Level 3	53.690 (-)	8.899 (-)	57.771 (-)	8.008
Level 4	33.599 (-)	36.819 (-)	19.815 (+)	31.881
Level 5	111.632 (-)	107.358 (-)	110.286 (-)	98.212
Level 6	37.863 (-)	37.915 (-)	35.632 (=)	35.772

Results using the Jaccard's similarity measure (left) and by MSE (right)



Experimental Results with GREC2005 Dataset (continue)...

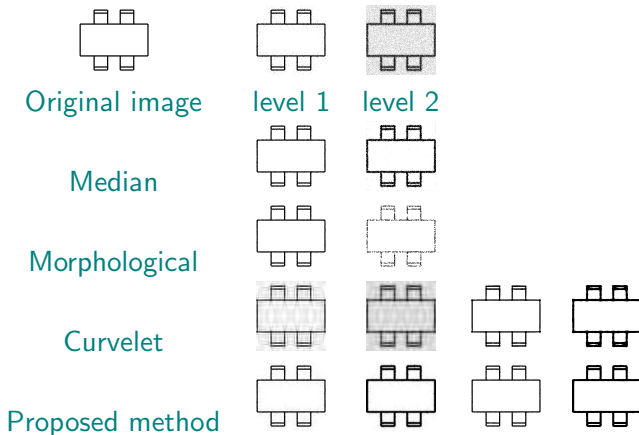


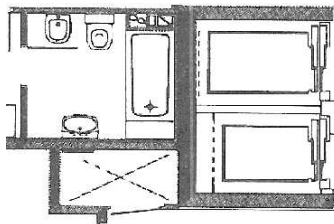
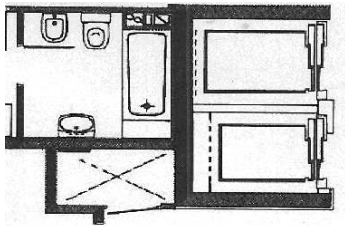
Figure: Results of denoising the noisy images with Kanungo model at levels 1 and 2 of degradation.

Experimental Results with Real Scanned Documents...

Images	Median	OC	Curvelet	Proposed method
1	0.5374	0.4640	0.4693	0.6834
2	0.6906	0.5165	0.6184	0.7534
3	0.5749	0.5158	0.5057	0.7421
4	0.6482	0.4879	0.5666	0.7454
5	0.6008	0.5056	0.5186	0.7458
6	0.6310	0.4409	0.5320	0.7145
7	0.6018	0.5030	0.5244	0.7580
8	0.6066	0.4814	0.5199	0.7381
9	0.6487	0.4662	0.5638	0.7456
10	0.6203	0.4603	0.5377	0.7317
11	0.6946	0.4832	0.5987	0.7685
12	0.7264	0.4494	0.6274	0.7733
Average	0.6320 (-)	0.4812 (-)	0.5485 (-)	0.7416

Table: The obtained results comparing our proposed method with the structural similarity SSIM index.

Experimental Results with Real Scanned Documents...

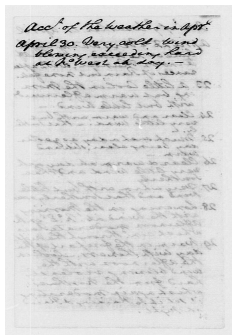
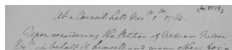


Experimental results with DIBCO 2009 dataset...

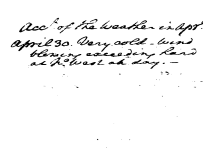
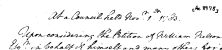
Images	Median	OC	Curvelet	Our approach
1	0.6420	0.6926	0.7054	0.9528
2	0.4628	0.5155	0.5156	0.9784
3	0.5115	0.5315	0.5064	0.8648
4	0.4595	0.4946	0.4692	0.8933
5	0.6953	0.7283	0.7441	0.9416
Average	0.5542 (-)	0.5925 (-)	0.5881 (-)	0.9261

Table: The obtained results with DIBCO 2009 dataset using SSIM measure.

Experimental results with DIBCO 2009 dataset...



(a)



(b)

Figure: (b) Denoised documents got by our approach after binarization using Otsu's method.

- We apply the sparse representation over the learned dictionary to document noise removal
- We find the way to define the value of finite energy of the noise ϵ

We can remove the noise even if the model of noise is unknown

Text/Graphics Separation

- **Important:** improve the accuracy rate, speed up the recognition process, input for OCR system, ...
- **Some methods:** can be roughly categorized into: edge-based methods, connected component-based methods, stroke-based methods, texture-based methods, or others...

However, In general, these methods are not robust when dealing with documents containing dense information

Thus, we proposed an approach using multi-learned dictionaries

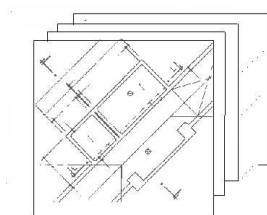
Multi-Learned Dictionaries: Motivation

- We have been strongly motivated by the good performance of MCA method

*The representation of text candidates in the **dictionary** for texts are sparse but not sparse enough in the **dictionary** for graphics*

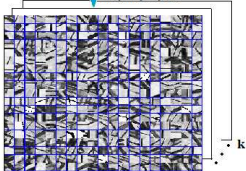
- We developpe the above assumption following 2 directions:
 - **The first one:** Using learned dictionary instead of using pre-defined dictionary $\Rightarrow$ better adapted to the characteristics of signal, thus reducing the overlap between them, or increasing the performance of text extraction
 - **The second one:** Using multi-learned dictionaries instead of only two dictionaries $\Rightarrow$ overcome the shortcoming about the size of signal

Text/Graphics Learned Dictionaries

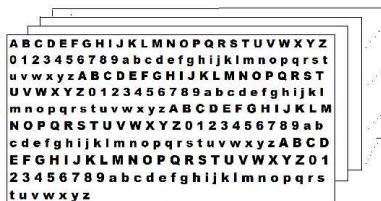


K-SVD

$W_1, W_2, \dots, W_k$

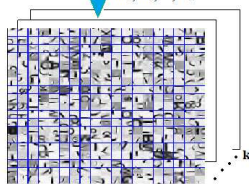


Graphic dictionary B_1



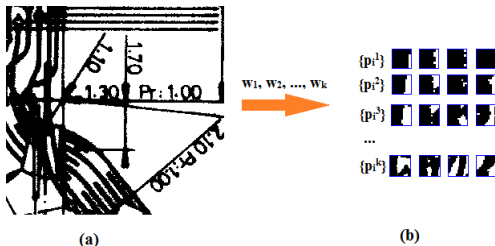
K-SVD

$W_1, W_2, \dots, W_k$



Text dictionary A_1

Text Regions Extraction by Learned Dictionary



- For each set of patches $\{p_i^k\}_i$ apply OMP algorithm to solve the equation following:

$$\bar{q}_{\Phi,i}^k = \arg \min_{q_{\Phi,i}^k} \|p_i^k - \Phi q_{\Phi,i}^k\|_2 \text{ s.t. } \|q_{\Phi,i}^k\|_0 \leq T_k$$

- Φ stands for A_k and B_k

Text Regions Extraction by Learned Dictionary

- Calculating the reconstruction error of p_i^k in text/graphic learned dictionary

$$\epsilon_{\Phi,i}^k = \|p_i^k - \Phi \bar{q}_{\Phi,i}^k\|_2$$

- Comparing two reconstruction errors $\epsilon_{A_k,i}^k$ and $\epsilon_{B_k,i}^k$
 - If ($\epsilon_{A_k,i}^k \leq \epsilon_{B_k,i}^k$), it means the representative of p_i^k in text dictionary A_k are sparse and not sparse (or at least not as sparse) in the graphic dictionary B_k . As the result, p_i^k will be considered to the text patch
- All same-sized patches in one category are merged to make a corresponding text or graphic layers.
- k text and graphic layers are combined into the final text/graphic layer by using the logical operations

$$y^T = \text{AND}(y_T^k) \text{ and } y^G = \text{OR}(y_G^k)$$

Postprocessing: Learn Thresholds

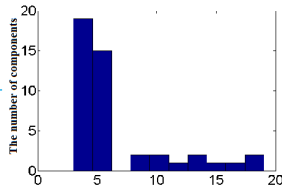
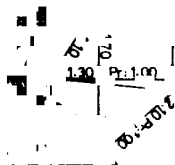
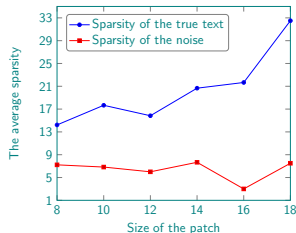
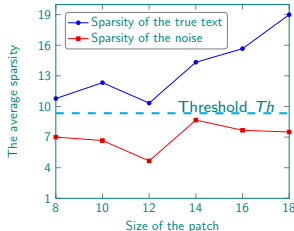


Image before the post processing step

The average sparsity

Image after the post processing step



Experimental Validation

Img	Nb. ch.	Hoang <i>et al</i>	Tombre <i>et al</i>	Our method	Dic. 8×8	Dic. 16×16
Doc 1	63	53	58	61	24	50
Doc 2	92	70	71	85	38	78
Doc 3	93	77	81	86	32	83
Doc 4	121	104	104	114	53	111
Doc 5	31	22	7	23	6	19

Table: Performance evaluation with $T_k = 16; 32$ for $\sqrt{s_k} = 8; 16$, respectively

Comment:

- Our method provides the best results for each document
- Using multi-dictionary provides better results than each dictionary used separately

Main contributions in our work for text/Graphics separation is that:

- We present an approach using multi-learned dictionaries for separation text/Graphics from technical documents
- We propose an original way to set the sparse thresholds automatically

Retrieving Similar Symbols

- **The goal:** giving a query symbol, retrieve all similar symbols and documents where it appears
- **Important:** Practical applications such as human interaction with information systems, retrieving image based on content, image classification or object recognition
- **Retrieval Symbol System:** includes 2 phases:
 - **Symbol description:** defines symbol based on pixel primitives, contours, closed regions, connected components or shape skeleton, partial relation between symbol primitives (e.g. lines, arcs), etc...
 - **Symbol retrieval:** comparing/matching descriptions of symbols;

Our work: Building vector model using sparsity in learned dictionary

Shape Context & Shape Context of Interest Point

- Shape Context (SC): describes an object based on the relationship between its contours points expressed in log-polar coordinates
- Shape Context of Interest Point (SCIP) is an adaptation of the SC defined only around interest points.

Learned Dictionary of SCIPs

- Dictionary A is learned by K-SVD algorithm with training database is SCIPs $\{\bar{h}_i\}_{i=1}^S$ computed from a set of N training symbols

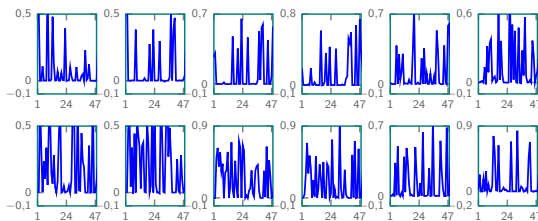


Figure: The 12 atoms (columns) of learned dictionary built from SCIP descriptors of symbols in GREC2003 database.

$$\text{K-SVD}(\{\bar{h}_i\}_{i=1}^S) \Rightarrow (A, \{x_i\}_{i=1}^S)$$

Visual Vector Construction

A training symbol n has r_n interest points, or r_n SCIP $\{h_i^n\}_{i=1}^{r_n}$ and $\{x_i^n\}_{i=1}^{r_n}$

- Compute characteristic vectors $\{v_i^n\}_{i=1}^{r_n}$ of $\{h_i^n\}_{i=1}^{r_n}$ based on $\{x_i^n\}_{i=1}^{r_n}$:

$$x_i^n = (\alpha_1, 0_2, \dots, \alpha_{p-1}, \alpha_p, 0_{p+1}, \dots, \alpha_q, \alpha_{q+1}, 0_{q+2}, \dots, 0_M)$$

$$v_i^n = (1_1, 0_2, \dots, 1_{p-1}, 1_p, 0_{p+1}, \dots, 1_q, 1_{q+1}, 0_{q+2}, \dots, 0_M)$$

- Define *tf-idf* factors adapted to the sparse representation by

$$tf_{k,n} = \frac{\sum_{i=1}^{r_n} v_i^n(k)}{\max_s \sum_{i=1}^{r_n} v_i^n(s)} \text{ and } idf_k = \log \frac{N}{\#\{n = 1, \dots, N | tf_{k,n} \neq 0\}}$$

- Calculate $w^n(k) = tf_{k,n} \times idf_k$ being the k -th element of the visual vector w^n for symbol n

$$n \implies w^n = (w^n(1), w^n(2), \dots, w^n(k), \dots, w^n(M))$$

Retrieval Symbol

For a query symbol s_i^q , we

- Compute its SCIPs $\{\bar{h}_i^q\}_i$
- Use OMP algorithm and learned dictionary A to compute sparse representations of $\{\bar{h}_i^q\}_i$
- Construction of the visual vector w_i^q of s_i^q based on these sparse representations
- Determine the similarity of a query symbol s_i^q and symbols in a database by the distance between two vectors w_i^q and w^n :

$$\text{distance}(w_i^q, w^n) = \frac{\langle w_i^q, w^n \rangle}{|w_i^q| \times |w^n|}$$

Experimental Validation

We have designed **three groups** of experiments that have allowed us to judge the performance of the sparse vector model approach

- **The first group** is to find the best parameters of the symbol retrieval system
- **The second group** aims at evaluating the performance of the SCIP descriptor, comparing to other shape descriptors. In addition, we have compared its performance to the k-means algorithm if it is used to learn the dictionary
- **The third group** seeks to evaluate the retrieval system in a more realistic configuration in which some symbol classes does not exist in the training set

Three datasets including GREC2003, CVC, FRESH are used

Experimental Validation: Studying of Parameters

Three parameters are studied including the dimension of SCIP descriptor (L), the approximation error (ϵ) and the size of visual dictionary (M). To avoid an exhaustive search $M = 512$ is fixed for all the experiments

	0.01	0.02	0.03	0.04	0.05	0.06	0.07	0.08	0.09	0.1	0.2
(12, 3)	0.533(-)	0.591(-)	0.595(-)	0.607(=)	0.611(=)	0.606(=)	0.615(=)	0.612(=)	0.560(-)	0.603(-)	0.602(-)
(12, 4)	0.345(-)	0.491(-)	0.556(-)	0.596(-)	0.605(=)	0.602(-)	0.613(=)	0.614(=)	0.609(=)	0.620(+)	0.106(-)
(12, 5)	0.290(-)	0.417(-)	0.558(-)	0.574(-)	0.597(-)	0.607(=)	0.608(=)	0.610(=)	0.604(=)	0.607(=)	0.614(=)
(16, 3)	0.421(-)	0.538(-)	0.580(-)	0.594(-)	0.598(-)	0.601(-)	0.604(-)	0.601(=)	0.603(-)	0.595(-)	0.610(=)
(16, 4)	0.308(-)	0.433(-)	0.542(-)	0.582(-)	0.599(-)	0.602(=)	0.612(=)	0.613(=)	0.606(-)	0.603(-)	0.619(=)
(16, 5)	0.211(-)	0.347(-)	0.504(-)	0.568(-)	0.584(-)	0.596(-)	0.593(-)	0.608(=)	0.584(-)	0.610(=)	0.616(=)

Table: Average AUC-PR values for the rotation and scale dataset.

Dataset	L	ϵ	Av. time (s)/query	Av. AUC-PR
GREC2003	48 (12×4)	0.1	1.380	0.6199 (+)
CVC	60 (12×5)	0.08	0.831	0.3604 (+)
FRESH	36 (12×3)	0.02	20.824	0.4018 (+)

Table: Best values for the datasets.

Experimental Validation: Invariance & Robustness

Original 	0.590	0.537	0.494	0.430	0.411	0.405	0.383	0.374	0.371	0.354
Rotation (90 degree) 	0.507	0.486	0.444	0.436	0.420	0.409	0.387	0.383	0.371	0.343
Scale (0.5) 	0.553	0.505	0.440	0.431	0.431	0.428	0.427	0.365	0.362	0.358
R (90 degree) and S (0.5) 	0.499	0.482	0.452	0.430	0.389	0.379	0.371	0.365	0.363	0.351
Original 	0.683	0.669	0.603	0.597	0.583	0.561	0.550	0.473	0.455	0.404
Rotation (90 degree) 	0.684	0.683	0.607	0.595	0.579	0.577	0.575	0.490	0.468	0.414
Scale (0.75) 	0.638	0.634	0.607	0.600	0.569	0.568	0.556	0.461	0.429	0.367
R (90 degree) and S (0.3) 	0.531	0.499	0.499	0.499	0.494	0.487	0.406	0.387	0.328	0.315

Table: Retrieval symbols on CVC dataset when request is rotated, scaled.

Experimental Validation: Invariance & Robustness

	SC	SCIP	SIFT	SIFT+sparse	Our Approach
Rotation and Scaling	0.088	0.548	0.153	0.213	0.620
Distortion	0.699	0.761	0.447	0.497	0.773
Deformation and Degradation	0.220	0.292	0.203	0.234	0.457
CVC	0.002	0.220	0.021	0.029	0.360
FRESH	0.301	0.286	0.355	0.443	0.402

Table: Retrieval effectiveness with AUC-PR values in different datasets

Comment: Constructing a vector model from the sparse representations of descriptors provides better results than using cluster algorithms like k-means

Experimental Validation: Unseen Symbols

Dataset	Descriptor + sparse	25%	50%	75%	100%
GREC2003	SCIP	0.610 (=)	0.592 (=)	0.610 (=)	0.620
	SIFT	0.211 (=)	0.203 (=)	0.209 (=)	0.213
CVC	SCIP	0.295(-)	0.306(-)	0.340(-)	0.360
	SIFT	0.024 (-)	0.027 (=)	0.028 (=)	0.029
FRESH	SCIP	0.386 (=)	0.387(=)	0.397(=)	0.402
	SIFT	0.429(-)	0.440(=)	0.436(=)	0.443

Table: Average of the AUC-PR values considering several percentages of training set size.

Comment: our approach is robust even if symbols are not in the learning set

Main contributions in our work for symbol recognition is that:

- We learned a dictionary of descriptors, thus
 - obtaining the characteristic of the invariance of sparse representation
 - overcoming the limitation of sparsity with high-dimensional signal
- We adapt tf-idf factors to sparsity,
 - Our proposed sparse vector model provides better results than using cluster algorithms like k-means

Introduction

- **The goal:** Identifying regions likely to contain a graphical symbol within graphics-rich documents
- **Some approaches:** the method based on a pseudo 2-D Hidden Markov Model, vectorization approaches, and structural approaches
- **Proposed method:** uses sparsity over learned dictionary of local descriptors.
 - **The off-line process:** dictionary is learned from a set of local descriptors computed over interest points of documents
 - **The on-line process:** symbol retrieval
 - The *recall* step: uses an indexing technique to efficiently retrieve regions of interest
 - The *refining* step: builds vector models of regions of interest based on the sparsity and the tf-idf approach adapted to the sparsity

Extension of SCIP for Documents

- Extension of SCIP (ESCIP) is SCIP descriptor calculated inside the neighbourhood region
- The neighbourhood region is an ellipse defined with the centre at interest point and the semi-major axis, semi-minor axis are decided depending on the scale in which this interest point is detected

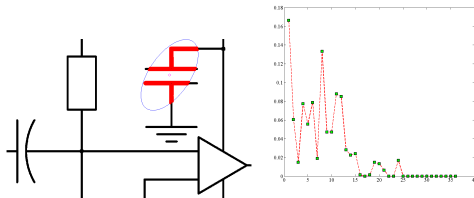
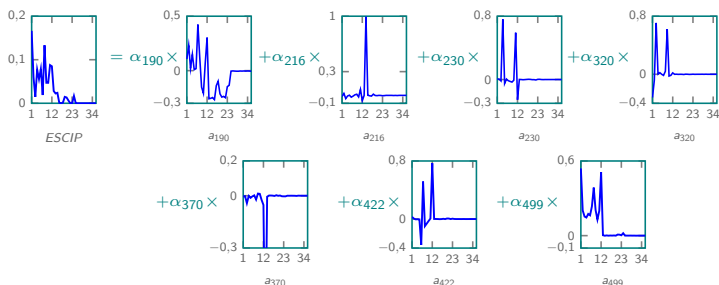


Figure: An example about ESCIP descriptor at interest point p_j^i of document

Learned Dictionary of ESCIPs

Dictionary A is learned by K-SVD algorithm with training database is ESCIPs computed from a set of documents



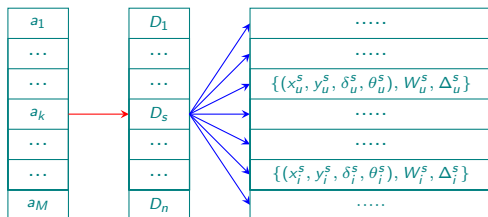
$$h_j^i \Rightarrow \bar{x}_j^i = (0, \dots, \alpha_{190}, \dots, \alpha_{216}, \dots, \alpha_{230}, \dots, \alpha_{320}, \dots, \alpha_{370}, \dots, \alpha_{422}, \dots, \alpha_{499}, \dots, 0) \in \mathbb{R}^M$$

$$\text{KSVD}(\{h_j^i\}_{i,j}) \Rightarrow (A, \{\bar{x}_j^i\}_{i,j})$$

Document Indexing

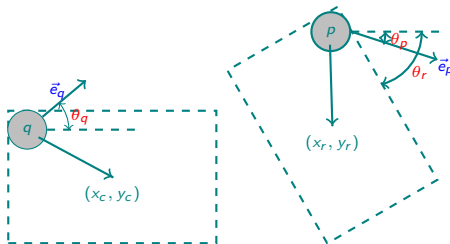
The inverted file structure is composed of two elements:

- **The vocabulary:** is the set of *visual words* of A
- **The occurrences:** each *visual word* is assigned to interest points having this word in their sparse representations, corresponding documents, their groups of visual words as well as coefficients in their representations



For example, if $h_i^s = \alpha_1^s \times a_1 + \alpha_k^s \times a_k + \alpha_t^s \times a_t + \alpha_l^s \times a_l$ then $W_i^s = \{a_1, a_k, a_t, a_l\}$ and $\Delta_i^s = \{\alpha_1^s, \alpha_k^s, \alpha_t^s, \alpha_l^s\}$

Location Symbol: Symbol Recall



$$\begin{pmatrix} x_r \\ y_r \end{pmatrix} = \frac{\delta_p}{\delta_q} G_{\theta_p - \theta_q} \begin{pmatrix} x_c - x_q \\ y_c - y_q \end{pmatrix} + \begin{pmatrix} x_p \\ y_p \end{pmatrix}$$

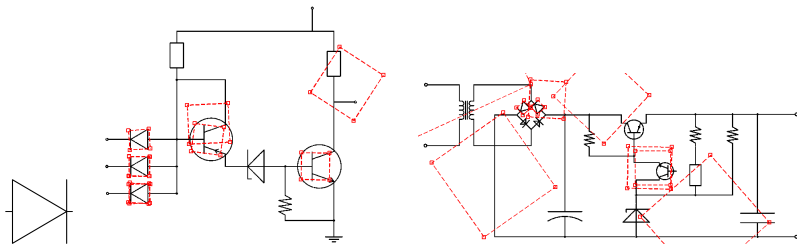
where $G_{\theta_p - \theta_q}$ is the rotation matrix, and the width w_r , the height h_r and the orientation θ_r of the region are given by:

$$h_r = h \times \frac{\delta_p}{\delta_q}; \quad w_r = w \times \frac{\delta_p}{\delta_q}; \quad \theta_r = \theta_q - \theta_p$$

Location Symbol: Symbol Recall

Two interest points p and q holds the *similarity property* at level $c \in (0, 1]$ if the following inequality is satisfied:

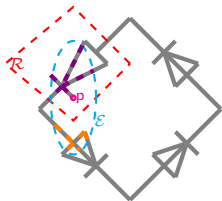
$$c \times |W_q| \leq |W_p \cap W_q| \leq |W_q|, \text{ with } c \in (0, 1]$$



Location Symbol: Symbol Refining

In the symbol refining step, we propose to

- calculate descriptors $h_p^{\mathcal{R}}$ only in regions of interest $\mathcal{R}$,
- build the vector model $v_{\mathcal{R}}$ of $\mathcal{R}$ based on all sparsity of $\{h_p^{\mathcal{R}}\}_p$ in A



- $OMP(A, \{h_p^{\mathcal{R}}\}_p) \Rightarrow \{\bar{x}_p^{\mathcal{R}}\}_p$

- Adapted *tf-idf* to the sparsity over A

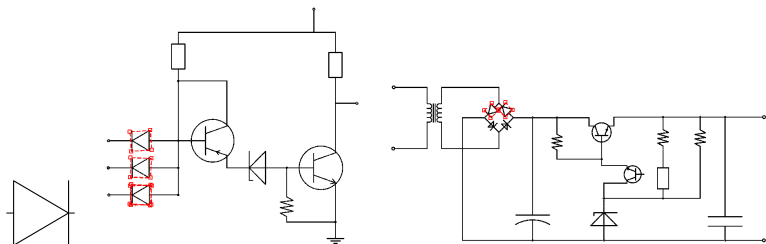
$$tf^{\mathcal{R}}(k) = \frac{\sum_{p \in \mathcal{R}} \bar{x}_p^{\mathcal{R}}(k)}{\sum_{j=1}^M \sum_{p \in \mathcal{R}} \bar{x}_p^{\mathcal{R}}(j)} \text{ and } idf(k) = \log\left(\frac{N}{l_k}\right)$$

- $v_{\mathcal{R}} = tf^{\mathcal{R}} \times idf$

- compare vector model v_q of query symbol q and vector model $v_{\mathcal{R}}$

$$distance(v_q, v_{\mathcal{R}}) = \frac{\langle v_q, v_{\mathcal{R}} \rangle}{|v_q| \times |v_{\mathcal{R}}|}$$

Location Symbol: Symbol Refining



Example of one request symbol and corresponding candidate regions after refining step

Experimental Validation

We evaluate our algorithm on 15 images of the SESYD dataset

Symbol	Retrieval measures			
	Proposed Approach		Nguyen <i>et al</i>	
classes	Precision (%)	Recall (%)	Precision (%)	Recall (%)
	100	100	100	100
	100	100	33.33	100
	54.55	52.63	50	42.85
	68.00	89.47	33.33	100
	100	100	100	100
	90	84.38	73.68	73.68
Average	85.43	87.74	65.06	86.09

	classes					
images						
image 1	8.6534	11.4800	12.8461	2.6261	21.4594	4.9100
image 2	8.6961	11.3504	12.8563	2.5374	21.3855	4.8034
image 3	8.6377	11.4096	12.8259	2.6336	20.7571	4.8560
image 4	8.7181	11.3959	12.8552	2.6078	20.9517	4.7784
image 5	8.6534	11.4260	12.8515	2.5890	20.7913	4.8589
image 6	8.7157	11.3998	12.8658	2.6142	20.9933	4.8065
image 7	8.6566	11.3706	12.8402	2.6295	20.9692	4.8377
image 8	8.7488	11.4102	12.8510	2.6098	21.0592	4.7998
image 9	8.6646	11.3915	12.8696	2.6343	20.9809	4.8657
image 10	8.7111	11.3695	12.8404	2.5645	20.9928	4.8055
image 11	8.6586	11.4164	12.8500	2.6580	21.0074	4.8675
image 12	8.7442	11.3737	12.8369	2.6196	21.2620	4.8178
image 13	8.6554	11.4137	12.8398	2.6101	20.8022	4.8580
image 14	8.7265	11.3629	12.8730	2.6024	21.0608	4.8028
image 15	8.6317	11.4065	12.8353	2.6136	20.5692	4.8669
Average	8.6848	11.3984	12.8558	2.610	20.9361	4.8357

Comment: Precision and recall rates increase in most case. The computing time is still high

- We adapt our proposed method for symbol recognition to symbol spotting
- We propose the two steps architecture including *recall* and *refining steps* in which
 - In *recall step*: the sparsity of descriptor is used to find the couple of interest points holding *similarity property*
 - In *refining step*: tf-idf factors are adapted to sparsity with the purpose of filtering out some interest regions

Conclusions and futur research

1 Our contributions:

- Document noise removal: *find the way to define the value of finite energy of the noise ϵ*
- Text regions extraction: *developing the assumption of MCA with multi-resolution learned dictionaries*
- Symbol recognition: *proprosing the original way to construct a sparse vector model*
- Spotting symbols: *proprosing the way to spot symbol based on sparse representation*

2 Future research:

- Speed up the proposed de-noising method
- Combination of sparse representations to other descriptions
- Spotting symbols in real documents

Thanks for your attention